Project :- Feature Engineering on Employee Dataset

Problem Statement

Real-world datasets are often incomplete, inconsistent, and not suitable for machine learning.

The goal of this project is to:

- Transform raw employee data into meaningful features
- Apply advanced feature engineering techniques
- Prepare high-quality dataset for future modeling

Objectives

- Handle missing values effectively
- Treat outliers
- Perform feature transformation
- Encode categorical variables
- Create new meaningful features
- Scale numerical features
- Select important features

Dataset Description

Dataset contains 1000+ employee records with:

- Demographic details
- Job-related information
- Salary

Complete Feature Engineering Pipeline

Step 1: Import Libraries

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

Step 2: Load Dataset

In [2]:
```
df = pd.read_csv("employee_data.csv")
df.head()
```

Out[2]:

	Name	Age	Gender	Department	Experience	City	Joining_Date	Salary
0	Emp_1	59.0	Female	Finance	3	Mumbai	11-01-2016	57999
1	Emp_2	49.0	Male	Sales	11	Mumbai	14-10-2012	84590
2	Emp_3	35.0	Male	Finance	7	Pune	22-12-2012	78181
3	Emp_4	28.0	Female	Sales	1	Bangalore	14-09-2009	35988
4	Emp_5	NaN	Male	HR	12	Delhi	10-02-2012	105112

Step 3: Initial Data Analysis

In [ ]:
```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Name          1000 non-null   object
 1   Age           950 non-null    float64
 2   Gender        1000 non-null   object
 3   Department    1000 non-null   object
 4   Experience    1000 non-null   int64
 5   City          1000 non-null   object
 6   Joining_Date  1000 non-null   object
 7   Salary        1000 non-null   int64
dtypes: float64(1), int64(2), object(5)
memory usage: 62.6+ KB
```

Out[ ]:
```
Name             0
Age             50
Gender           0
Department       0
Experience       0
City             0
Joining_Date     0
Salary           0
dtype: int64
```

Step 4 Check Missing Values

In [4]:
```
df.isnull().sum()
```

Out[4]:
```
Name             0
Age             50
Gender           0
Department       0
Experience       0
City             0
Joining_Date     0
Salary           0
dtype: int64
```

Step 5 Handling Missing Values (Imputer)

In [5]:
```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')

df['Age'] = imputer.fit_transform(df[['Age']])
```

Explanation

- SimpleImputer fills missing values
- Strategy = mean (you can use median also)

In [6]:
```
df.isnull().sum()
```

Out[6]:
```
Name            0
Age             0
Gender          0
Department      0
Experience      0
City            0
Joining_Date    0
Salary          0
dtype: int64
```

Step 6 Encoding

- Machine Learning models cannot understand text (categorical data)
- They only understand numbers
- Encoding Converts Categorical Values Into Numerics Values

Label Encoding

In [7]:
```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])
```

Explanation

- Male → 1, Female → 0
- Used for binary categories

One Hot Encoding

In [8]:
```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(drop='first', sparse_output=False)

encoded = ohe.fit_transform(df[['Department','City']])

encoded_df = pd.DataFrame(encoded, columns=ohe.get_feature_names_out())

df = pd.concat([df, encoded_df], axis=1)

df.drop(['Department','City'], axis=1, inplace=True)
```

Step 7 Time-Based Feature Engineering

In [10]:
```
df['Joining_Date'] = pd.to_datetime(df['Joining_Date'], format='%d-%m-%Y')

df['Year'] = df['Joining_Date'].dt.year
df['Month'] = df['Joining_Date'].dt.month

df['Years_at_Company'] = 2025 - df['Year']
```

Explanation

We extracted:

- Year
- Month
- Experience in company
- Dates → useful numeric features

Step 8 Feature Scaling

- Features have different ranges
- Model gives more importance to bigger values
- So we scale to same range

Min-Max Scaling

In [11]:
```
from sklearn.preprocessing import MinMaxScaler

minmax = MinMaxScaler()

df[['Age']] = minmax.fit_transform(df[['Age']])
```

- Range: 0 to 1

Standardization

In [12]:
```
from sklearn.preprocessing import StandardScaler

std = StandardScaler()

df[['Experience']] = std.fit_transform(df[['Experience']])
```

- Mean = 0, Std = 1

Robust Scaling

In [13]:
```
from sklearn.preprocessing import RobustScaler

robust = RobustScaler()

df[['Years_at_Company']] = robust.fit_transform(df[['Years_at_Company']])
```

- Best when outliers present

Step 9 Skewed Data Handling

In [14]:
```
print(df['Salary'].skew())
```

```
-0.0331064067483959
```

Apply Log Transformation

In [15]:
```
import numpy as np

df['Salary_log'] = np.log1p(df['Salary'])
```

Explanation

- Skewed data → uneven distribution
- Log transform → makes it normal
- Helps ML model perform better

Final Dataset

In [16]:
```
df.head()
```

Out[16]:

	Name	Age	Gender	Experience	Joining_Date	Salary	Department_HR	Depart
0	Emp_1	1.000000	0	-1.137601	2016-01-11	57999	0.0	
1	Emp_2	0.736842	1	0.240057	2012-10-14	84590	0.0	
2	Emp_3	0.368421	1	-0.448772	2012-12-22	78181	0.0	
3	Emp_4	0.184211	0	-1.482015	2009-09-14	35988	0.0	
4	Emp_5	0.514183	1	0.412264	2012-02-10	105112	1.0	

Now dataset is:

- Clean
- Encoded
- Scaled
- Transformed

Step 10: Export Clean Dataset

In [17]:
```
df.to_csv("employee_cleaned.csv", index=False)
```

Key Insights

- Missing values handled properly using sklearn
- Encoding converts categorical → numeric
- Time features improve understanding
- Scaling prevents bias
- Log transformation fixes skewness

Conclusion

This project demonstrates complete Feature Engineering pipeline using sklearn.

We applied:

- Imputation
- Encoding
- Time-based features
- Scaling techniques
- Skewness handling